

mixed_up.c 단순화 결과 보고

입력 정보

- 대상 파일명: mixed_up.c
- 테스트 대상 함수: target_a, target_b, target_c
- 총 입력 개수: 각 함수당 15개 입력 인자
- 입력 데이터 타입: 모두 uint64_t
- 주요 연산 변수: x, y
- 나머지 변수: z, a, b, c, d, e, f, t, k, var1, var2, dummy(난독화를 위한 무의미 변수)
- 입력 함수 설명: 실제 연산은 x, y를 중심으로 수행되며 나머지 변수는 난독화 패턴에만 사용됨.

```
uint64_t target_a(uint64_t x , uint64_t y , uint64_t z , uint64_t a , uint64_t b ,
                uint64_t c , uint64_t d , uint64_t e , uint64_t f , uint64_t t ,
                uint64_t k , uint64_t var1 , uint64_t var2 , uint64_t dummy )
```

전처리 및 단순화 조건

항목	설정 값 및 설명
사용 도구	ProMBA (Program-level MBA Simplifier)
실행 명령어	./promba.exe -timeout 4000.0 -rulefile ./no.rules -norule -sygus ./simbas.sh -synth2 ./synth.sh -Isolver ./simbad mixed_benchmark/mixed_up.c
-timeout	4000.0초, 전체 단순화 과정의 제한 시간
-rulefile	./no.rules (초기에는 rule 없이 실험 진행)
-norule	미리 정의된 MBA Rule을 적용하지 않음
-sygus	Syntax-Guided Synthesis 수행 스크립트 (simbas.sh)
-synth2	2차 프로그램 합성기를 통한 후보 식 생성 (synth.sh)
-Isolver	논리 조건 검증을 위한 SMT solver 래퍼 (simbad)
최대 허용 AST 높이	3 (단순화된 식의 Abstract Syntax Tree 최대 깊이 제한)
입력 파일명	mixed_up.c
대상 함수 수	3개 (target_a, target_b, target_c)
단순화 목표	난독화 수식을 원래 수식($x - y$, $x + y$, $x ^ y$)으로 복원
입력 변수 수	15개 (모두 uint64_t형 정수)
난독화 기법	무의미 연산, 항등식, 연산자 재정렬 등을 조합한 MBA 표현 사용

중간 결과 및 난독화 식(before deobfuscation)

```

/* Generated by CIL v. 1.7.3 */
/* print_CIL_Input is true */

#line 55 "/usr/include/stdint.h"
typedef unsigned long uint64_t;
#line 5 "mixed_up.c"
uint64_t target_a(uint64_t x, uint64_t y, uint64_t z, uint64_t a, uint64_t b,
                  uint64_t c, uint64_t d, uint64_t e, uint64_t f, uint64_t t,
                  uint64_t k, uint64_t var1, uint64_t var2, uint64_t dummy )
{
    {
#line 6
        return (x - y) + (((x ^ x) - (x - x))) + (((x & ~x) | (x - x))) +
            (((~x) + 1) + x)) + (((y ^ y) - (y - y))) + (((x & x) - (x & x)))
            + (((x - x) ^ (y - y))) + (((x + 0) - x)) + (((0 + y) - y)) + (0 *
            (x - x)) + (((x | 0) - x)) + (((y & 0) + (0 & y))) + (((~(x & 0)) -
            (~x & 0))) + (((x ^ 0) - x)) + (((x + 0) - (0 + x))) + (((0 - 0))
            ^ 0) + (((0 / 1) - (0 / 1))) + (((x - x) + (y - y))) + (((x & ~x) -
            (x & ~x))) + (((x | y) - (x | y))) + (((x ^ y) - (x ^ y))) + (((x +
            (~x + 1)) - 0)) + (((~y + 1) + y)) + (((x - x) ^ (x - x))) + (((0 *
            0) + (0 / 1))) + (((x | ~x) ^ (x | ~x))) + (((x & x) ^ (x & x))) +
            (((x - 0) - x)) + (((0 + 0) - (0 + 0))) + ((0) + (0)) + (((x - x) *
            (y - y)));
    }
}
#line 20 "mixed_up.c"
uint64_t target_b(uint64_t x, uint64_t y, uint64_t z, uint64_t a, uint64_t b,
                  uint64_t c, uint64_t d, uint64_t e, uint64_t f, uint64_t t,
                  uint64_t k, uint64_t var1, uint64_t var2, uint64_t dummy )
{
    {
#line 21
        return (x + y) + ((x ^ x) - (x - x)) + ((x & ~x) + (x - x)) + (((~x)
            + 1) + x) + (((y ^ y) - (y - y)) + ((y & ~y) + (y - y)) + (((x | ~x)
            - (x | ~x))) + (((x - x) ^ (y - y)) + ((x + 0) - x) + ((0 + y) - y)
            + (0 * (x - x)) + ((x | 0) - x) + ((y & 0) + (0 & y)) + (((~(x & 0))
            - (~x & 0))) + ((x ^ 0) - x) + ((x + 0) - (0 + x)) + (((0 - 0)) ^ 0)
            + (((0 / 1) - (0 / 1))) + ((x - x) + (y - y)) + (((x & ~x) - (x & ~x
            ))) + (((x | y) - (x | y))) + (((x ^ y) - (x ^ y))) + ((x + (~x + 1)
            ) - 0) + (((~y) + 1) + y) + ((x - x) ^ (x - x)) + (((0 * 0) + (0 / 1
            ))) + (((x | ~x) ^ (x | ~x))) + (((x & x) - (x & x))) + (((x - 0) -
            x)) + (((0 + 0) - (0 + 0))) + (((x * 1) - x)) + (((1 * y) - y)) + ((
            (x & 1) - (x & 1))) + (((x | 1) - (x | 1))) + (((~0) - (~0)));
    }
}
#line 30 "mixed_up.c"
uint64_t target_c(uint64_t x, uint64_t y, uint64_t z, uint64_t a, uint64_t b,

```

```

uint64_t c , uint64_t d , uint64_t e , uint64_t f , uint64_t t ,
uint64_t k , uint64_t var1 , uint64_t var2 , uint64_t dummy )
{
{
#line 31
return (x ^ y) + (((x ^ x) - (x - x))) + (((x & ~x) + (x - x))) + (((
~x) + 1) + x)) + (((y ^ y) - (y - y))) + (((y & ~y) + (y - y))) + ((
(x | ~x) - (x | ~x))) + (((x - x) ^ (y - y))) + (((x + 0) - x)) + (((
0 + y) - y)) + (0 * (x - x)) + (((x | 0) - x)) + (((y & 0) + (0 & y))
) + (((~(x & 0)) - ~(x & 0))) + (((x ^ 0) - x)) + (((x + 0) - (0 +
x))) + (((0 - 0) ^ 0) + (((0 / 1) - (0 / 1))) + (((x - x) + (y - y))
) + (((x & ~x) - (x & ~x))) + (((x | y) - (x | y))) + (((x ^ y) - (x
^ y))) + ((x + (~x + 1)) - 0) + (((~y) + 1) + y) + (((x - x) ^ (x - x
))) + (((0 * 0) + (0 / 1))) + (((x | ~x) ^ (x | ~x))) + (((x & x) - (
x & x))) + (((x - 0) - x)) + (((0 + 0) - (0 + 0))) + (((x * 1) - x))
+ (((1 * y) - y)) + (((x & 1) - (x & 1))) + (((x | 1) - (x | 1))) + (
((~0) - (~0))) + (((x - x) * ((x | ~x) - (x | ~x))) + (((x + (0 * 1)
) - x)) + (((y - (0 / 1)) - y)) + (((x ^ 0) ^ (0 ^ x))) + (((~x) ^ (
~x) - 0)) + (((0 + 0) ^ (0 + 0))) + (((x * 0) + (y * 0))) + (((0 - 0
) - (0 - 0))) + (((x + 0) + (0 - x)) - 0) + ((x & 0) - (0 | 0)) + (((
~x) & 0) + (0 ^ 0)) + (((x & 1) ^ (1 & x)) - ((x & 1) - (1 & x))) + (
((x | 2) - (x | 2)) + ((2 & x) - (2 & x))) + (((~x) + ~(~x))) - ((~
x) + ~(~x))) + (((x ^ 2) ^ (2 ^ x)) - ((x ^ 2) - (2 ^ x))) + (((
0 | 0) ^ (0 & 0)) - ((0 | 0) + (0 & 0)));
}
}

```

- 모든 함수는 수십 개의 무의미하거나 중복된 연산을 포함해 복잡한 수식을 구성
- 연산자 반복, 상수 간 계산, 의미 없는 XOR, AND, NEGATION 반복 포함
- 평균 AST 노드 수(OrigSize): 157 ~ 284

단순화 결과(after deobfuscation)

항목 (Metric)	target_a	target_b	target_c
함수 이름(Name)	target_a	target_b	target_c
단순화된 수식 결과(Result)	x - y	y + x	x ^ y
단순화 후 AST 크기(Size)	3	3	3
원래 AST 크기(OrigSize)	157	184	284
사용된 변수 수(VarNum)	2	2	2
다항식 여부(IsPoly)	false	false	false
원본 MBA 변화 정도(AlterOrig)	8	7	12
단순화 후 MBA 변화 정도(AlterDeob)	0	0	0
학습된 Rule 수(RuleNumber)	25	25	25
Rule 적용 횟수(RuleCount)	28	27	154

선형 간소화 감소량(LinearReduce)	113	146	181
합성 기반 감소량(SynthReduce)	4	9	60
Rule 기반 감소량(RuleReduce)	40	29	43
수행 시간(초)(Time(s))	11.0	19.0	26.0

Before vs After code

Function	Deobfuscation 전 (복잡한 원래 식 일부 요약)	Deobfuscation 후 (단순화된 결과)
target_a	return (x - y) + (((x ^ x) - (x - x)) + ... (AST 157)	return (x - y); (AST 3)
target_b	return (x + y) + ((x ^ x) - (x - x)) + ... (AST 184)	return (x + y); (AST 3)
target_c	return (x ^ y) + (((x ^ x) - (x - x)) + ... (AST 284)	return (x ^ y); (AST 3)

결과 분석

1. 정확도

- target_a, target_b, target_c 모두 예상대로 정확하게 단순화됨
 - target_a: $x - y$
 - target_b: $y + x$
 - target_c: $x ^ y$
- 이로써 복잡하게 작성된 MBA 표현식이 올바르게 복원되었음을 확인

2. 복잡도

함수 이름	OrigSize	평가
target_a	157	✅ 충분한 난독화
target_b	184	✅ 충분한 난독화
target_c	284	✅ 매우 높은 난독화

- 모두 OrigSize가 150이상으로, AST의 노드 수가 많은 복잡한 수식으로 구성되어 있음

3. 간소화 효과

함수	Size (단순화 후)	RuleCount	LinearReduce	SynthReduce	RuleReduce
A	3	28	113	4	40
B	3	27	146	9	29
C	3	154	181	60	43

- 모든 함수가 최종적으로 AST Size가 3으로 줄어들었음
- target_c는 rule 적용 횟수가 154회로 가장 많아 가장 난해한 표현을 포함했음을 시사

4. 처리 시간

- target_c가 26초, target_b는 19초, target_a는 11초로 점차 증가하는 경향이 있음
- 이는 복잡도(OrigSize)와 rule 적용 횟수(RuleCount)에 따라 시간이 선형 이상으로 증가함을 보여줌